Karpathy Modelleri FPGA'de · teknik terimler sözlüğü

Nasıl kullanılır: Sunumda geçen bütün teknik terimler konu başlıklarına göre gruplandı; her grubun içinde alfabetik sıradadır. Terimler sunumdaki gibi İngilizce bırakıldı; parantez içinde Türkçe karşılığı, sonunda terimin geçtiği slaytlar (s.) var. Bir terim iki anlamda kullanılıyorsa (ör. LUT) iki anlamı da yazıldı.

---

1. Dil modelleri ve eğitim

activation (ara değer) — Bir katmanın çıkışında oluşan ve sonraki katmana giren sayılar. Weight'ler modelle sabit gelir; activation'lar her token için yeniden hesaplanır. Bizde INT8 (residual akış INT16). s. 12, 17

argmax — Bir vektördeki en büyük elemanın indeksi. Greedy üretimde bir sonraki token, logit'lerin argmax'ıdır. "argmax aynı" oranı: float ve integer modelin en olası token'da anlaştığı adımların oranı (stories260K'da %92). s. 17, 27

autoregressive generation (kendi çıktısıyla beslenen üretim) — Model bir token üretir, onu girişe ekler ve bir sonrakini üretir. Kartın üretim döngüsü tam olarak bunu yapar. s. 26–27

bias (sapma) — Linear katmanın çıkışına eklenen sabit. Bizde requant parametreleriyle birlikte parametre ROM'unda durur. s. 13, 18

checkpoint — Eğitilmiş bir modelin kaydedilmiş weight dosyası. stories260K, ailede başkasının (Karpathy'nin) eğittiği tek checkpoint. s. 9

context (bağlam) — Modelin bir token'ı tahmin ederken görebildiği geçmiş token sayısı. names modellerinde 16, kartta stories modellerinde 256. s. 9, 23

cross-entropy loss, test loss (çapraz entropi kaybı) — Modelin doğru token'a verdiği olasılığın eksi logaritmasının ortalaması; küçük olan iyidir. Birimi nat/token. Hiçbir şey bilmeyen bir model 27 harflik alfabede ln 27 ≈ 3,30 alır; gpt2'nin INT8 hâli 2,002. s. 17, 33, 36, 47

float, floating point, float32 (kayan noktalı sayı) — Bilgisayarda gerçel sayıların üs ve mantis ile tutulması. Eğitim float32 ile yapılır; FPGA tarafında hiç kullanılmaz. s. 11

greedy — Her adımda en olası token'ı seçmek (argmax). Rastgelelik yoktur; aynı başlangıç her zaman aynı metni verir. Kartta SW0 ile seçilir. s. 27, 30

inference (çıkarım) — Eğitilmiş modeli çalıştırmak. FPGA yalnızca inference yapar; eğitim ve QAT Mac'te. s. 4, 31

KL divergence (Kullback–Leibler ıraksaması) — İki olasılık dağılımının ne kadar farklı olduğunun ölçüsü. Knowledge distillation'da öğrencinin dağılımı öğretmenin dağılımına KL ile yaklaştırılır. s. 32

knowledge distillation (KD) (bilgi damıtma) — Küçük ya da düşük bitli bir "öğrenci" modelin, büyük ya da float bir "öğretmen" modelin çıktı dağılımını taklit ederek eğitilmesi. Düşük bitli QAT'de float model öğretmen. s. 32

language model (dil modeli) — Geçmiş token'lara bakarak bir sonraki token'ın olasılık dağılımını veren model. s. 4

logit — Modelin her token için ürettiği, softmax'tan önceki ham skor. Engine logit'leri INT16 olarak sampler'a verir. s. 26–28

nat — Doğal logaritmayla ölçülen bilgi birimi. Loss farkları nat/token olarak verildi (ör. stories260K'da INT8'in bedeli +0,022 nat). s. 17

parameter (parametre) — Modelin eğitimle öğrenilen sayılarının (weight ve bias) toplamı. Aile ~50 bin, stories260K ~260 bin parametre. s. 7, 9

QAT (quantization-aware training) (quantization'ı bilen eğitim) — Modeli, ileri yönde quantize edilmiş weight'lerle çalıştırarak eğitmek; model düşük bite uyum sağlar. s. 31–33

sampling (örnekleme) — Bir sonraki token'ı olasılık dağılımından rastgele seçmek. Temperature ve min-p ile ayarlanır. s. 27–28

softmax — Logit'leri olasılıklara çeviren fonksiyon: $p_k = e^{l_k} / \Sigma e^{l_j}$. Attention'da ve sampling'de kullanılır; bizde tamamen integer. s. 14, 28

temperature (τ) (sıcaklık) — Logit'leri τ'ya bölerek dağılımı keskinleştiren ($\tau < 1$) ya da yayan ($\tau > 1$) ayar. Kartta SW[3:1] ile 8 değerden biri. s. 28, 30

min-p — Olasılığı en olası token'ın belli bir oranından (ör. 1/32) küçük olan token'ları atan sampling kuralı. Saçma token'ları keser. s. 28

token — Modelin işlediği en küçük birim. names modellerinde bir harf (27 token), stories260K'da bir kelime parçası (512 token). s. 7, 9

training (eğitim) — Weight'lerin loss'u azaltacak şekilde güncellenmesi. Tamamen Mac'te, float ile. s. 4

vocabulary (V) (sözcük dağarcığı) — Modelin tanıdığı token'ların sayısı: names'te 27, TinyStories'te 512. s. 9, 27

weight (ağırlık) — Linear katmanların öğrenilmiş katsayıları. Bizde INT8, binary modellerde 1 bit; hepsi BRAM'de. s. 12, 20

2. Model mimarileri ve yapı taşları

attention (dikkat) — Her token'ın geçmiş token'lara içeriğe göre ağırlık vererek bilgi toplaması. Query, key ve value vektörleriyle hesaplanır. s. 8, 14, 23

bag of words (BoW) — makemore'daki basit model: geçmiş token'ların causal ortalamasını alır. s. 7

bigram — Yalnızca son token'a bakan model: 27 × 27'lik bir tablo. Ailenin en basiti ve en hızlısı (saniyede 318 bin token). s. 7, 45

causal — Bir pozisyonun yalnızca kendinden önceki pozisyonları görebilmesi. KV cache'i mümkün kılan özellik. s. 7, 23

decoder-only transformer — Yalnızca geçmişe bakan, metin üreten transformer türü (GPT ailesi). Ailedeki gpt2, microgpt, llama, nanochat.

embedding — Token id'sini öğrenilmiş bir vektöre çeviren tablo; matematikte one-hot çarpımı, pratikte tablo okuması. Engine'de EMB op'u. s. 8, 21

GELU — Transformer'larda kullanılan yumuşak bir activation; x · Φ(x). gpt2'de; bizde 256 satırlık tabloyla. s. 16

GQA (grouped-query attention) — Birkaç query head'inin aynı key/value head'ini paylaşması; KV cache küçülür. llama ve stories260K'da 8 query head'e 4 KV head (stories260K). s. 7, 19

GRU (gated recurrent unit) — Kapılarla (gate) neyi hatırlayıp neyi unutacağını seçen recurrent model. names'te en iyi loss (1,956). s. 7, 47

head — Attention'ın paralel çalışan alt birimlerinden biri; her biri farklı bir ilişkiye bakabilir. Bizde head boyutu 8. s. 19, 23

KV cache — Geçmiş token'ların key ve value vektörlerinin saklanması; yeni token gelince eskileri yeniden hesaplanmaz. stories260K'da 80 KB BRAM. s. 6, 20, 23

LayerNorm — Vektörü ortalaması sıfır, varyansı bir olacak şekilde normalize edip gain (γ) ve bias (β) uygulayan katman. gpt2'de. s. 15

learned positional embedding (öğrenilen pozisyon) — Her pozisyon için öğrenilmiş bir vektörün token embedding'ine eklenmesi. gpt2 ve microgpt'de. s. 7

Linear — Matris çarpımı ve bias'tan oluşan katman, y = Wx + b. Bir token işlenirken matris-vektör çarpımına (GEMV) iner. s. 8, 22

MHA (multi-head attention) — Birden çok head'li standart attention. s. 7

MLP — (1) Transformer block'undaki iki Linear'lı ileri besleme katmanı; (2) makemore'daki Bengio (2003) tarzı model: son 16 token'ın embedding'lerini birleştirir. Sunumda "mlp" modeli ikinci anlamda. s. 7

QK-norm — Query ve key vektörlerinin attention'dan önce normalize edilmesi; nanochat'te. Norm'un cycle payını %14'ten %33'e çıkarır. s. 15, 24

query, key, value (q, k, v) — Attention'ın üç vektörü: query "ne arıyorum", key "bende ne var", value "ne veriyorum". Score = q · k. s. 14, 19, 23

ReLU, ReLU² — max(0, x) ve onun karesi. microgpt ReLU, nanochat ReLU² kullanır. s. 7

residual (stream) (artık bağlantı) — Bir block'un çıkışının girişine eklenmesi; bilgi ana bir "akış" üzerinde taşınır. Bizde INT16 ve GEMV'nin "+= residual" modu. s. 19, 21

RMSNorm — Ortalamayı çıkarmadan yalnızca kök ortalama kareye bölen normalization. llama2.c (gain'li), microgpt ve nanochat (parametresiz). s. 7, 15

RNN (recurrent neural network) — Geçmişi bir state vektöründe taşıyan model; her token'da state güncellenir. s. 7

RoPE (rotary position embedding) — Pozisyonu, query ve key vektörlerini pozisyona bağlı bir açıyla döndürerek kodlayan yöntem. llama ve nanochat'te; engine'de ROPE op'u, sin/cos tablosuyla. s. 7, 19, 21

SiLU, sigmoid (σ), tanh — Yaygın activation fonksiyonları. SiLU = x · σ(x) (SwiGLU'da), σ ve tanh GRU'nun kapılarında. Hepsi 256 satırlık tabloyla. s. 16

state (durum) — Recurrent modellerin geçmişi taşıdığı vektör. Engine'de COPY, GRUUPD ve HRESET op'ları. s. 8

SwiGLU — İki Linear'ın çıkışlarından birinin SiLU'dan geçirilip ötekiyle eleman bazında çarpıldığı MLP türü. llama'da; engine'de MUL op'u. s. 7, 19

transformer — Attention ve MLP block'larından oluşan mimari. Ailede gpt2, microgpt, llama, nanochat ve stories260K. s. 7

3. Quantization ve integer arithmetic

accumulator (akümülatör) — Çarpım-toplamların biriktiği geniş register. INT8 GEMV'de 32 bit; binary GEMVB'de 20 bit yeter ($|acc| \le 128 \cdot n_in$). s. 13, 22, 35

bit-exact (bit bit aynı) — İki hesabın her bitinin aynı olması; "yaklaşık aynı" değil. Donanım ile Python'daki integer referans bit-exact. s. 2, 37–38

calibration (kalibrasyon) — Activation scale'lerinin örnek verilerden seçilmesi. Bizde $|x|$'in %99,99 yüzdeliği. s. 12, 18

clip, sat, saturation (kırpma, doyma) — Sınırları aşan değerin sınıra çekilmesi: INT8'de −128 … 127. s. 13, 16

fixed point (sabit nokta) — Kesirli sayının bir tamsayı ve sabit bir 2'nin kuvveti ile tutulması; ör. $M \approx m_0 \cdot 2^{-sh}$. s. 13, 28

folding (katlama) — Matematiksel olarak eşdeğer bir dönüşümle bir işlemi başka bir işlemin içine gömmek; ör. LayerNorm'un γ ve β'sı sonraki Linear'ın weight ve bias'ına. s. 15, 18

INT4, INT8, INT16 — 4, 8 ve 16 bitlik işaretli tamsayılar. INT8: −128 … 127. s. 11–13

integer-only (yalnızca tamsayı) — Hiçbir adımda floating point kullanmayan hesap. Bizde softmax, norm ve sampling dahil her şey. s. 2, 11

isqrt (tamsayı karekök) — $\lfloor \sqrt{Q} \rfloor$'yu bit bit hesaplayan sıralı devre; norm biriminde. s. 15

LUT (lookup table, tablo anlamında) — Bir fonksiyonun bütün değerlerinin önceden hesaplanıp tabloya yazılması. INT8 girişli her fonksiyon 256 satırlık bir tablodur (GELU, SiLU…); exp için 2^(−k/256) tablosu. FPGA'deki LUT ile karıştırmayın (bkz. bölüm 5). s. 14, 16

m_0, sh — Requant çarpanının tamsayı kısmı ve kaydırma miktarı: $M \approx m_0 \cdot 2^{-sh}$, $m_0 \in [2^{15}, 2^{16})$. Örnek: M = 0,0013703 → sh = 25, m_0 = 45 979. s. 13

MAC (multiply-accumulate) (çarp-topla) — $a \cdot b$'yi bir akümülatöre ekleme işlemi; matris-vektör çarpımının temel adımı. Bir DSP48 cycle başına bir MAC yapar. s. 11

per-channel, per-tensor scale (kanal başına, tensör başına scale) — Weight'lerde her çıkış satırının kendi scale'i; activation'larda bütün tensöre tek scale. s. 12

percentile (yüzdelik) — Değerlerin belli bir oranının altında kaldığı eşik. %99,99 yüzdeliği seçmek uç değerleri kırpar, kalanlara daha ince çözünürlük kazandırır. s. 12

PTQ (post-training quantization) (eğitim sonrası quantization) — Eğitilmiş modeli yeniden eğitmeden quantize etmek. int8 yöntemimiz PTQ. s. 32

quantization (nicemleme) — Gerçel sayıyı scale × tamsayı olarak yazmak: q = clip(round(r/S), −127, 127). Örnek: S = 0,00211 için w = 0,0372 → q = 18. s. 12

requant (requantization) (yeniden nicemleme) — INT32 akümülatörü bir sonraki katmanın INT8 scale'ine çevirmek: y = sat((acc · m_0 + 2^{sh-1}) ≫ sh). Örnek: acc = 5 321 → 7. s. 13

rounding (yuvarlama) — Kaydırmadan önce 2^{sh-1} eklemek "yarıyı yukarı" yuvarlamadır. s. 13

Samuelson eşitsizliği — N elemanlı her vektörde $|x_i - \mu| \leq \sigma\sqrt{(N-1)}$. Normalize edilmiş çıkışın büyüklüğünü veriden bağımsız sınırlar; norm çıkışı asla doymaz. s. 15

scale (S) — Quantization'da tamsayıyı gerçel değere çeviren çarpan: r̂ = S · q. s. 12

shift (≫) (kaydırma) — Bir sayıyı 2'nin kuvvetine bölmenin donanımdaki karşılığı; bedava sayılır. s. 13–14

xorshift32 — Kaydırma ve XOR'la çalışan basit bir sözde rastgele sayı üreteci; sampling'in rastgele sayısı. s. 28

4. Low-bit weight'ler: Larq ve LittleBit

Bi-Real Net, MagnitudeAwareSign — Binary ağlar için bir yöntem; weight'i işaretine indirip kanal başına ortalama mutlak değeri scale yapar. Larq'ta MagnitudeAwareSign olarak var; bizim bin yöntemimiz. s. 32

binary weight (ikili weight) — Weight'in yalnızca işaretiyle (±1) ve bir scale ile tutulması: w ≈ a · sign(w). Çarpma toplama-çıkarmaya döner; bellek 8 kat küçülür. s. 31, 35

BPW (bits per weight) (weight başına bit) — Bir katmanı temsil etmek için harcanan toplam bitin weight sayısına oranı; scale'ler de dahil. INT8 ≈ 8, binary 1 + 16/d_in. s. 33–34

Dual-SVID — LittleBit'in başlangıç yöntemi: weight matrisinin SVD'sinden işaret matrisleri ve scale vektörleri çıkarır. Tek başına yetmez; QAT şart. s. 32

Larq — Binary ve ternary sinir ağları için TensorFlow/Keras kütüphanesi. Quantizer'larını (SteSign, SteTern, MagnitudeAwareSign) PyTorch'ta yeniden yazdık. s. 31–32

LittleBit — Lee vd. (NeurIPS 2025): weight matrisini diag(h) · sign(U) · diag(ℓ) · sign(V)ᵀ · diag(g) olarak, yani low-rank binary factor'lar ve scale vektörleriyle temsil edip 1 bitin altına iner. s. 31–35

low-rank, rank (r) (düşük rank) — Bir matrisi iki ince matrisin çarpımı olarak yazmak; r iç boyut. LittleBit'te rank, min(d)'nin %r'si (lb25 = %25). s. 32, 34

lb25p1, lb25p2… — Bizim LittleBit adlandırmamız: rank %25, 1 ya da 2 yol (path). mlp-lb25p1 = tek yollu, rank %25 mlp. s. 33, 36

path, residual path (yol) — LittleBit'te ilk yaklaşımın hatasını ikinci bir binary factor çiftiyle düzeltmek. 2 yol bit bütçesini ikiye katlar. s. 34

sign (işaret fonksiyonu) — x ≥ 0 ise +1, değilse −1. s. 31

STE (straight-through estimator) — Türevi olmayan sign gibi bir fonksiyonu eğitebilmek için, geri yönde gradient'i (|w| ≤ 1 iken) olduğu gibi geçirmek. s. 32

ternary (üçlü) — Weight'in {−a, 0, +a} değerlerinden biri olması. Larq SteTern (TWN). Binary'den fazla bit harcadı, ondan iyi değildi. s. 32–33

TWN (ternary weight networks) — Ternary weight için yöntem; eşik 0,7 · ort|w|. s. 32

5. FPGA, kart ve donanım kavramları

BRAM (block RAM), RAMB36, RAMB18 — Çipteki bellek blokları; XC7A100T'de 135 adet 36 kilobitlik blok (~607 KB). Her RAMB36 iki RAMB18'e bölünebilir. Weight'ler, KV cache ve tablolar burada. s. 5, 20, 44

CARRY4 — FPGA'de hızlı toplama için ayrılmış carry zinciri bloğu. LUT'larla kurulan çarpıcılar uzun carry zincirleri oluşturur. s. 43–44

clock, MHz, cycle (saat, döngü) — Senkron devrenin vuruşu ve iki vuruş arası. 25 MHz'te bir cycle 40 ns. Hız = clock / (cycle/token). s. 6, 45

combinational path (kombinasyonel yol) — İki register arasındaki, register'sız mantık zinciri; bir cycle içinde tamamlanmak zorunda. s. 43

DSP48E1 — Çipteki hazır çarpıcı-toplayıcı (25 × 18 bit): cycle başına bir MAC. XC7A100T'de 240 adet. Açık kaynak akışta Yosys hatası yüzünden kullanılmadı. s. 5, 40, 44

FF (flip-flop) — Tek bitlik register; ara sonuçları bir cycle'dan ötekine taşır. 126 800 adet. s. 5, 44

FPGA (field-programmable gate array) — Devresi üretimden sonra yapılandırılan çip; LUT, FF, BRAM ve DSP yapı taşlarından oluşur. s. 5

LUT (look-up table, FPGA anlamında) — FPGA'nin temel mantık bloğu: 6 girişli küçük bir doğruluk tablosu, her mantık fonksiyonu bunlarla kurulur. XC7A100T'de 63 400 LUT (nextpnr'da 126 800 "LUT bel", her LUT6 iki yuva). s. 5, 44

LUT bel — nextpnr'ın saydığı LUT yuvası; her fiziksel LUT6'da iki yuva var. stories260K 34 643 yuva kullanır (%27). s. 44

LUTRAM — LUT'lardan yapılan küçük RAM; okuması asenkron. Engine'in vektör tamponları LUTRAM'de ve okuma kritik yolun parçası. s. 43

MMCM (mixed-mode clock manager) — 100 MHz osilatörden başka clock frekansları (50 ya da 25 MHz) üreten blok. s. 6

Nexys A7-100T — Digilent'in, Artix-7 XC7A100T çipli eğitim kartı; işlemcisi yok. Hedef kartımız. s. 5

pipeline (boru hattı) — Uzun bir işi register'larla aşamalara bölmek; her aşama bir cycle'a sığar. Engine'e pipeline eklemek 50 MHz'in yolu. s. 43, 50

QSPI flash — Kart açılınca bitstream'i kendiliğinden yükleyen flash bellek. s. 5

register — Bir değeri clock vuruşunda saklayan flip-flop grubu. s. 43

senkron tasarım, tek clock — Bütün register'ların aynı clock'la çalışması; clock domain geçişi sorunu yok. Zamanlama analizini yeterli kılan özellik. s. 6, 37

USB-JTAG — Bitstream'i bilgisayardan karta yükleme arayüzü. s. 5

VGA, piksel hızı — Monitör sinyali; 640 × 480 @ 60 Hz, 25 MHz piksel hızı. Kart metni buradan gösterir. s. 29

XC7A100T, Artix-7 — AMD (Xilinx) Artix-7 ailesinden çipimiz: 63 400 LUT, 126 800 FF, 135 BRAM, 240 DSP. s. 5

7-segment, LED, RGB LED, anahtar (SW), düğme (BTNC, BTNU, BTND, CPU_RESET) — Kartın giriş/çıkışları: anahtarlar ayarları, düğmeler koşuları yönetir; LED16 kart üstü kontrolün sonucunu gösterir (yeşil OK, kırmızı FAIL). s. 30

debounce (sıçrama giderme) — Mekanik düğmenin titreşimini süzmek; bir basış 10 ms boyunca kararlı kalmalı. s. 26

enable — Bir devrenin her cycle değil, belli cycle'larda çalışmasını sağlayan sinyal; VGA piksel hızı ana clock'tan böyle türetilir. s. 6

6. Engine, compiler ve bizim tasarımımız

board_pkg, zoo_pkg — Her model için üretilen SystemVerilog paketleri: boyutlar, bellek derinlikleri, beklenen hash'ler, OPS_USED. s. 25

broadcast (yayınlama) — Bir değerin aynı anda birçok hatta verilmesi; GEMV'de giriş x_i 8 hatta yayınlanır. s. 22

compiler (derleyici) — PyTorch modelini mikrokoda ve bellek imajlarına çeviren yazılım (sw/zoo/ir.py, quant.py, export_hw.py). s. 18

detokenizer — Token numarasını karakterlere çeviren devre (iki ROM tablosu). s. 26, 29

engine, zoo_core — Bu projede sıfırdan tasarlanan, microcode ile programlanan inference accelerator core'u; bütün modelleri aynı devreyle çalıştırır (rtl/zoo/zoo_core.sv). "Engine" genel bir donanım terimidir (inference engine, compute engine); bizim engine'imizin adı zoo_core'dur. s. 21

fetch, decode — Program belleğinden bir op kelimesini okuyup alanlarına ayırmak. s. 21

FSM (finite state machine) (durum makinesi) — Durumlar arasında geçen kontrol devresi; üretim döngüsü G_STEP, G_WAIT, G_MAX, G_SUM, G_SEL, G_PICK durumlarından oluşur. s. 27

GEMV (general matrix-vector multiply) — Matris-vektör çarpımı; bir token işlenirken her Linear katmanın işi. Engine'de 8 hatlı INT8 birim. s. 22, 24

GEMVB — Bizim 1-bit GEMV birimimiz (opcode 18): 64 toplama-çıkarma hattı, DSP yok. Binary ve LittleBit modelleri burada çalışır. s. 35

golden reference, IntRunner (altın referans) — Donanımın bit bit aynı üretmek zorunda olduğu Python'daki integer program. s. 18

hash, kart üstü kontrol — Üretilen token'lardan hesaplanan özet: h ← 31h + tok. Python'un bitstream'e gömdüğü değerle karşılaştırılır. s. 30

IR (intermediate representation) (ara temsil) — Modelin op'ların düz bir listesi olarak yazılmış hâli; PyTorch ile farkı 10^{-4}'ün altında. s. 18

lane (hat) — Paralel çalışan aynı hesap birimlerinden biri. GEMV'de 8, GEMVB'de 64 hat. s. 22, 35

lm_gen — Üretim döngüsü, sampler, detokenizer, terminal ve kart üstü kontrolü içeren modül. s. 26

mikrokod, op, opcode, prog.mem — Engine'in yürüttüğü komut listesi; her op bir işlem (GEMV, NORM, ATTN...). gpt2'de 43, stories260K'da 73 op. s. 19

.mem dosyaları, $readmemh — Bellek içeriklerini tutan metin dosyaları (prog, weights, params, tables, luts, wbin) ve onları Verilog'da BRAM'e başlangıç içeriği olarak okuyan komut. s. 18, 20

op türleri — EMB/EMB8/EMB16/EMBWIN/TABLOGIT (embedding ve tablo okuma), GEMV, GEMVB, NORM (LayerNorm/RMSNorm), ATTN, KVSTORE (KV cache'e yaz), ROPE, MUL (eleman bazlı çarpma), REQ (requant), SCALE, COPY, GRUUPD, HRESET (recurrent state), BOW (causal ortalama), END. s. 8, 21

output-stationary dataflow — Çıkışların hatlarda sabit kalıp girişlerin aktığı düzen; weight adresi basit bir sayaç olur. s. 22

SCALE — Bizim op'umuz (opcode 17): elemanı INT16 bir gain ile çarpıp requant eder; binary weight'i bozmadan norm gain'ini taşımak için. W·diag(γ) = diag(a) · S · diag(g). s. 35

shadow bank (gölge banka) — Bitmiş akümülatörlerin kopyalandığı register'lar; requant onlardan okurken hatlar sonraki bloğa başlar. s. 22

SPECIALIZE, OPS_USED — Modelin kullanmadığı birimlerin sentezde atılması; OPS_USED, kullanılan op'ların bit maskesi. bigram'da engine 5 855'ten 1 529 LUT'a iner. s. 25

terminal — 80 × 28'lik metin alanını yöneten devre: kelime kaydırma (word wrap), scroll, durum satırı. s. 29

vektör tamponu (vector buffer, vrf) — Engine'in ara vektörleri tuttuğu 11 tampon; residual INT16, diğerleri INT8. s. 21

vga_text, font ROM, text RAM, bin2bcd — Ekran devreleri: metin RAM'indeki karakterleri font ROM'uyla piksele çevirir; bin2bcd sayıları onluğa çevirir. s. 29

daktilo modu (typewriter) — Üretimi saniyede ~25 token'a yavaşlatan mod (SW6); gözle izlemek için. s. 30, 45

7. Verification ve test

2-state, 4-state simulation — 2-state simülasyon her biti 0 ya da 1 sayar (Verilator); 4-state simülasyon bilinmeyen (X) ve yüksek empedans (Z) değerleri de temsil eder (Icarus). X, reset'i unutulmuş register'ları yakalar. s. 39

CI (continuous integration) — Her değişiklikte testleri otomatik çalıştıran düzen (scripts/nexys_ci.sh, GitHub Actions). s. 37

corner case, directed test (köşe durumu, yönlendirilmiş test) — Elle yazılmış uç durum testleri; ör. koşu sırasında basılan düğme. 16 adet. s. 39

fuzz, random testing (rastgele test) — Rastgele girişlerle çok sayıda test: 655 koşu, ~121 bin token. s. 39

netlist — Sentez aracının ürettiği kapı listesi (LUT, FF, BRAM bağlantıları). Netlist simülasyonu araç hatalarını yakalar. s. 40

pin signature (pin imzası) — Her cycle'da bütün pinlerin değerinden hesaplanan özet; RTL ile netlist'i cycle cycle karşılaştırmak için. s. 38

RTL (register-transfer level) — Devrenin register'lar ve aralarındaki mantıkla tarif edildiği seviye; SystemVerilog kodumuz. s. 37

simulation, cycle-accurate (simülasyon) — Devrenin bilgisayarda clock clock çalıştırılması. Tam kart simülasyonunda testbench bir kullanıcı gibi düğmelere basar. s. 38

testbench — Simülasyonda devreyi süren ve çıkışlarını kontrol eden kod (tb/tb_nexys.sv). s. 38

X değeri — 4-state simülasyonda bilinmeyen değer; reset'siz register'ların belirtisi. s. 39

8. Synthesis, place and route ve timing araçları

bitstream — FPGA'nin bütün yapılandırma bitlerini içeren dosya (.bit, XC7A100T'de 3,8 MB): LUT tabloları, kablo anahtarları, BRAM içerikleri. Model weight'leri de bitstream'in içinde. s. 20, 41

chipdb — nextpnr'ın kullandığı çip veritabanı (yerleşim yerleri, kablolar, gecikmeler); Project X-Ray verisinden üretilir. s. 41

critical path (kritik yol) — Devredeki en uzun gecikmeli yol; Fmax'ı belirler. stories260K'da 36,7 ns, %73'ü kablolama. s. 43

FASM — Açılacak FPGA özelliklerinin metin listesi; nextpnr yazar, Project X-Ray bitstream'e çevirir. s. 41

Fmax — Devrenin zamanlamayı tuttuğu en yüksek clock frekansı: 1 / (en uzun yolun gecikmesi). gpt2-bin: 1 / 33,7 ns ≈ 29,7 MHz. s. 42

frames — FPGA yapılandırma belleğinin satırları; xc7frames2bit bunları .bit'e çevirir. s. 41

HeAP placer — nextpnr'ın analitik yerleştiricisi; ts_gpt2 ve ts_microgpt'de BRAM %85 doluyken ilerleyemedi. s. 42

MMCM / clock constraint, create_clock — Zamanlama analizine clock periyodunu söyleyen kısıt (XDC'de). s. 41

nextpnr (nextpnr-himbaechel) — Açık kaynak yerleşim ve yönlendirme aracı; Xilinx 7 serisi desteği himbaechel altında. s. 41

place and route (yerleşim ve yönlendirme) — Netlist'in çipteki fiziksel yerlere konması (place) ve aralarındaki kabloların seçilmesi (route). s. 41

Project X-Ray (prjxray) — Xilinx 7 serisinin bitstream formatını ve gecikmelerini topluluk olarak çözen proje; açık kaynak bitstream'in temeli. s. 41

routing delay (kablolama gecikmesi) — Sinyalin bir mantık bloğundan diğerine giderken kablolarda geçirdiği süre. s. 43

seed (tohum) — Yerleştiricinin rastgele kararlarının başlangıç değeri; Fmax'ı birkaç MHz oynatabilir (bow: 50,1 ve 44,0 MHz). s. 42

slack, pay — Yolun gecikmesi ile clock periyodu arasındaki fark. stories260K 25 MHz'te 3,3 ns pay. s. 43

static timing analysis (STA) (statik zamanlama analizi) — Simülasyon yapmadan her yolun gecikmesini toplayıp periyoda sığıp sığmadığını kontrol etmek. s. 37, 42

sv2v — SystemVerilog'u Verilog'a çeviren araç; 0.0.13 sürümünde signedness hatası bulduk. s. 40

synthesis (sentez) — RTL'i LUT, FF, BRAM ve DSP'lerden oluşan bir netlist'e çevirmek (Yosys). s. 41

Verilator, Icarus Verilog — Açık kaynak simülatörler: Verilator hızlı, 2-state; Icarus yavaş, 4-state. s. 38–39

Vivado — AMD/Xilinx'in resmî FPGA aracı; macOS'ta çalışmadığı için kullanılmadı. s. 41, 49

XDC, PACKAGE_PIN, LOC — Pin atamalarını ve clock kısıtlarını içeren kısıt dosyası; nextpnr'daki hata yüzünden pinleri LOC ile yazdık. s. 41

Yosys, synth_xilinx, -nodsp — Açık kaynak sentez aracı ve Xilinx akışı; -nodsp çarpıcıları DSP yerine LUT'a koyar (DSP eşlemesindeki işaret hatası yüzünden). s. 40–41

9. Karpathy'nin depoları, modeller ve veri setleri

GPT-2 — OpenAI'ın 2019 modeli; ailedeki gpt2, onun block yapısı (LayerNorm, GELU, öğrenilen pozisyon). s. 7

Llama 2 — Meta'nın 2023 modeli; RMSNorm, SwiGLU, RoPE ve GQA. llama2.c onun sade C uygulaması. s. 7, 9

llama2.c — Karpathy'nin Llama 2'yi tek bir C dosyasında çalıştıran deposu; stories260K buradan. s. 7, 9

llm.c — Karpathy'nin GPT-2'yi C/CUDA ile eğiten deposu; hazır modelleri (124M) BRAM'e sığmaz. s. 49

makemore — Karpathy'nin karakter seviyesinde isim üreten modeller deposu: bigram, mlp, rnn, gru, bow, transformer. s. 7

microgpt — Karpathy'nin 2026'da yayımladığı ~200 satırlık, bağımlılıksız GPT gist'i; parametresiz RMSNorm, ReLU, bias yok. s. 7

minGPT, nanoGPT — Karpathy'nin GPT-2'yi sade PyTorch ile yazdığı depolar; ailedeki gpt2 ikisinin ortak block'u. nanoGPT'nin hazır checkpoint'leri BRAM'e sığmaz. s. 7, 49

nanochat — Karpathy'nin en yeni, uçtan uca sohbet modeli deposu; çekirdek mimarisi (RoPE, QK-norm, ReLU², GQA) ailede. s. 7

names.txt — makemore'un ~32 bin İngilizce isimden oluşan veri seti; 27 karakter. s. 7

stories260K — Karpathy'nin TinyStories'te eğittiği 260 bin parametreli Llama 2: 64 boyut, 5 layer, 8 head, 4 KV head, 512 token. s. 9

TinyStories — Küçük modellerin tutarlı metin üretebilmesi için çocuk hikâyelerinden oluşturulmuş veri seti (Eldan ve Li, 2023). s. 9

ts_* — Aynı mimarilerin bizim TinyStories'te ~260 bin parametreyle eğittiğimiz sürümleri (ts_gpt2, ts_gru, ts_llama, ts_microgpt, ts_nanochat). s. 9

gpt-nano — minGPT'nin en küçük yapılandırması (3 layer, 3 head, C = 48); ilk sunumdaki sıralama demosu. Bu kart tasarımında yok. s. 49–50